ESRS Project Specification

1. Introduction

Our project is aimed at developing a scalable, reliable, and modern forum application. The system will support multiple user roles and provide robust functionalities ranging from topic creation and messaging to moderation and administrative capabilities. Throughout the full software life-cycle, we will apply agile project management, test-driven development, CI/CD practices, and modern software design techniques such as containerization and orchestration.

2. Functional Requirements

2.1 User Stories

For every feature, we will follow the user story format:

As a [role], I want [action] so that [goal].

1. Guest

1.1 As a guest, I want to see categories so that I can explore discussions that interest me.
1.2 As a guest, I want to see topics and posts so that I can read discussions without needing an account.
1.3 As a guest, I want to search for topics and posts using fuzzy search so that I can find relevant discussions even if I don’t remember the exact keywords.
1.4 As a guest, I want to see the number of replies, last activity, number of views, and tags of a topic so that I can gauge the popularity and relevance of discussions.
1.5 As a guest, I want to see user statistics (number of posts, topics) so that I can understand their activity level in the forum.

2. User

2.1 As a user, I want to register using my email so that I can participate in discussions.
2.2 As a user, I want to log in with my credentials so that I can access my account and interact with the forum.
2.3 As a user, I want to confirm my email so that I am recognized as a confirmed user.
2.4 As a user, I want to change my password using email so that I can log in with my new password.
2.5 As a user, I want to change my profile picture so another user can see my new profile picture.
2.6 As a user, I want to send private messages to other users so that I can communicate with them directly.

2.7 As a user, I want to get a notification when I receive a private message so that I am aware of new messages.
2.8 As a user, I want to reply to private messages so that I can continue conversations with other users.
2.9 As a user, I want to create or add tags to my topics so that I can categorize my discussions effectively.
2.10 As a user, I want to create a topic with styled text and file attachments so that I can enhance the readability and depth of my posts.
2.11 As a user, I want to receive notifications when someone answers my topic so that I stay informed about responses.
2.12 As a user, I want to add my topic to a specific category when creating it so that it is organized properly.
2.13 As a user, I want to create posts by replying to topics so that I can engage in discussions.

3. Moderator

3.1 As a moderator, I want to modify a topic's category so that discussions are correctly organized.
3.2 As a moderator, I want to pin a topic to the top of the page so that important discussions remain visible.
3.3 As a moderator, I want to close a topic so that no further posts can be added when necessary.
3.4 As a moderator, I want to delete tags so that I can remove irrelevant or inappropriate tags from topics.
3.5 As a moderator, I want to delete posts so that I can remove inappropriate or off-topic content.
3.6 As a moderator, I want to delete topics so that I can remove discussions that violate community guidelines.
3.7 As a moderator, I want to restrict or ban users so that I can maintain a healthy community environment.

4. Administrator

4.1 As an administrator, I want to create and modify categories so that I can organize the forum effectively.
4.2 As an administrator, I want to change a user's role to a moderator so that I can delegate moderation responsibilities.
4.3 As an administrator, I want to see analytics (e.g., daily signups, pageviews, posts) so that I can track the forum's activity and engagement.

2.2 Actors and Their Capabilities

Guest

- **View & Search**:
  - Can view categories, topics, and posts.
  - Can perform a fuzzy search on topics and posts.
  - Can see metadata such as the number of replies, last activity, number of views, and tags.
  - Can view basic user information (e.g., number of posts and topics).

User

- **Account Management**:
  - Register and log in using an email-based account.
- **Communication**:
  - Send and answer private messages.
  - Receive notifications for new private messages.
- **Content Creation**:
  - Create topics with styled text and file attachments.
  - Add topics to specific categories.
  - Post responses (create posts) to topics.
  - Create or add tags to topics.

Moderator

- **Content & Community Management**:
  - Delete posts and topics.

- Move topics between categories.
- Modify a topic’s category.
- Pin topics to the top of a page.
- Close topics (to prevent further posts).
- Delete tags.
- Restrict, or otherwise manage users as needed.

Administrator

- **System Administration & Analytics**:
  - View a comprehensive dashboard panel.
  - Edit user roles (e.g., promote a user to moderator).
  - Create and modify categories.
  - Access analytics (e.g., daily sign-ups, page views, post counts) to monitor system health and user engagement.

3. Non-Functional Requirements

3.1 Documentation

- **Design Documents**: Detailed use cases, user stories, and design diagrams.
- **Code Documentation**: Inline code comments and external documentation.
- **Test Documents**: Comprehensive test cases and reports.
- **Development Documents**: Process documentation including meeting notes, sprint retrospectives, and a development blog.

3.2 Continuous Integration and Continuous Deployment (CI/CD)

- Establish a CI/CD pipeline using containerized stacks.

- Automate builds, testing (including unit, system, and integration tests), and deployments.

3.3 Caching

- Implement caching mechanisms to optimize performance.
- Evaluate caching strategies for dynamic and static content.

3.4 Scalability

- **Data and User Growth**: Handle a large number of users and data.
- **Functional Scalability**: Design the system so that new features (e.g., a mobile app) can be integrated with minimal changes to the backend.
- **Infrastructure Scalability**: Support increased loads via scalable container orchestration and load balancing.

3.5 Test-Driven Development (TDD)

- Develop tests before feature implementation.
- Ensure high test coverage across all components.
- Include pressure and load testing to ensure system reliability.

3.6 Reliability and Security

- **Security Measures**: Implement robust security protocols to protect user data.
- **Testability**: Use stress testing and load testing to validate performance.
- **Logging and Backup**: Implement comprehensive logging, backup, and redundancy strategies.

4. Scope of the Project

The ESRS project encompasses the development of an online forum system that includes:

- Multi-user management with four distinct roles (Guest, User, Moderator, Administrator).
- Features for content creation, communication, moderation, and administrative control.
- CI/CD and agile development practices to ensure high quality and quick iterations.
- Scalability and reliability to support growing user bases and increased data loads.
- Documentation and automated testing to support maintainability and future evolution.

5. Technology stack

Component	Technology Used	Justification
Web Server	Nginx	Fast and reliable web server, that supports load balancing.
Backend	Django (Python)	Secure and scalable backend framework with rapid development features.
Frontend	React.js	Efficient and scalable UI framework with a component-based structure.
Caching	Redis	Enhances performance by caching frequently accessed data.
Background Tasks	Celery	Handles asynchronous processing for scheduled and background tasks.
Database	MySQL / PostgreSQL	Reliable relational database with ACID compliance.

CI/CD	GitHub Actions	Automates testing, building, and deployment.
Containerization	Docker	Standardized environments across all stages of development.
Cloud Hosting	AWS / Google Cloud	Provides scalable infrastructure and managed services.

6. Development Methodology

We will use the **Scrum agile methodology** with one-week iterations and daily stand-up meetings. The project is scheduled over six weeks, providing a buffer for unforeseen challenges.

We will adopt a local-first development approach: develop and deploy locally for efficient testing, then deploy to the cloud for production.

Trello will be used for managing the team backlog and tracking tasks. Scrum Artifacts (Product Backlog, Sprint Backlog, Increment, and Burndown Charts) will be maintained as part of our development team blog.

6.1 Scrum Team Structure

Role	Team Member
Product Owner	Dr. Hsi-ming Ho
Scrum Master	Erfan Ghoreishi
Frontend Developer	Yergali Kabash
Frontend Developer	Srinivasan Ramesh

Backend Developer & Designer	Erfan Ghoreishi
Backend Developer	Hugo Whitworth
Documentation Lead	Nareshkumar Arumugam

6.2 Week-by-Week Plan

Week 1:

- Requirement analysis and system design.
- Sketching architectural diagrams and designing the backend.
- Learning test-driven development and containerization.

Week 2:

- GitHub repository setup.
- Setup of the CI/CD pipeline and containerization (using Docker, etc.).
- Initial exploration of Kubernetes for container orchestration.

Week 3:

- Development of the authentication system.
- implementation of the basic UI for login/registration.
- Continued work on CI/CD procedures.

Week 4:

- Development of debugging tools.

- Implementation of main forum functionalities (categories, topics, posts).
- Completion of initial UI enhancements.

Week 5:

- Development of notification and private messaging systems.
- Implementation of caching strategies.
- Exploring UI/UX enhancements for main forum functionalities (if applicable).

Week 6:

- Integration testing (including load and security testing).
- Final debugging and performance tuning.
- Adjustments to container orchestration and load balancing for enhanced scalability.

6.3 Version Control Strategy

We will use Git for version control, hosted on Git Hub. Our team will follow a modified Git Flow branching model, simplifying it to reduce complexity while maintaining structured development and collaboration.

7. Conclusion

This document outlines the complete specification for our forum project, detailing the system's functional and non-functional requirements, development methodology, and technology stack. It serves as a structured roadmap, guiding our team through the design, implementation, and deployment phases. By adhering to the outlined specifications, we aim to deliver a scalable, reliable, and efficient forum system that meets user expectations while ensuring maintainability and future growth. This document will remain a key reference throughout the project, helping us stay aligned with our goals and development standards.